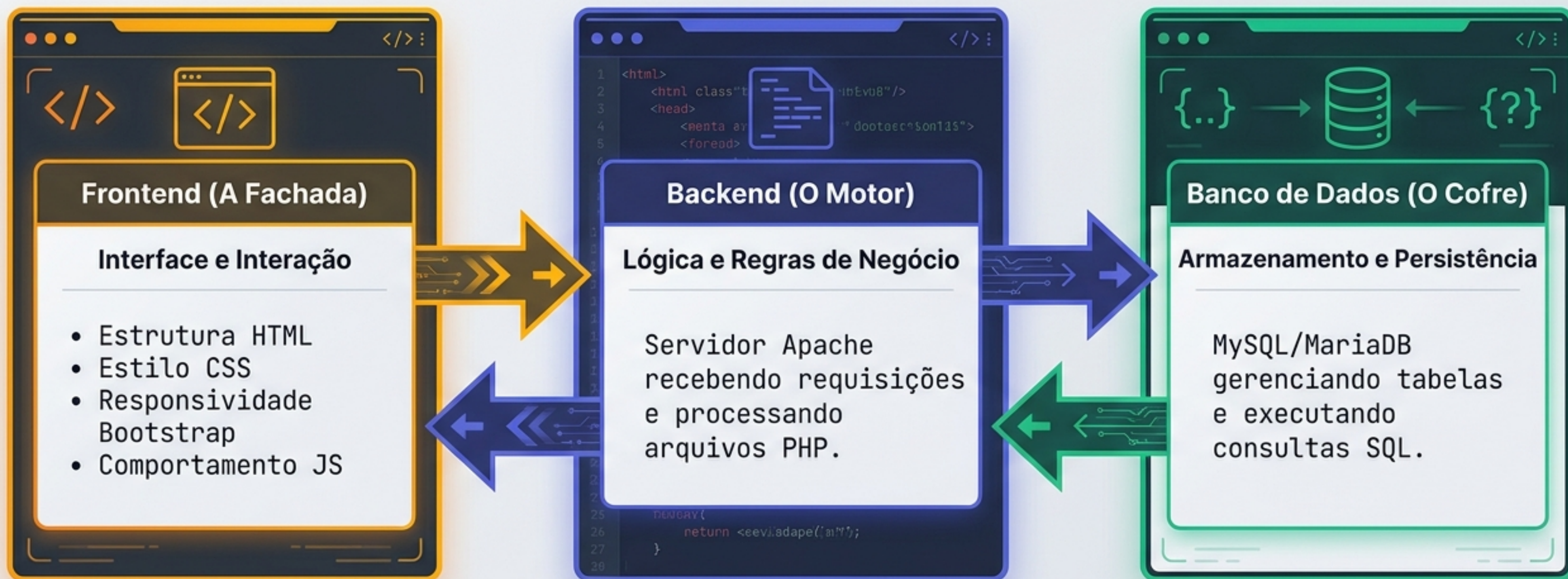


Desenvolvimento Web Completo

Frontend, Backend e Banco de Dados com
PHP, MySQL, HTML, CSS, JS e Bootstrap.

Uma abordagem prática da Engenharia de Software.
Código é mais que sintaxe; é a materialização de
soluções para problemas reais.



O sucesso de um sistema não está apenas na **tecnologia escolhida**, mas em como essas três camadas **conversam entre si**.

⚙️ Ambiente XAMPP (Localhost)



Apache (Servidor Web)

Recebe requisições **HTTP**, executa **PHP** e retorna **páginas**.



MySQL/MariaDB (Banco de Dados)

Armazenamento persistente e execução de **SQL**.



PHP (Linguagem Backend)

Processamento, regras de negócio e **integração**.



phpMyAdmin (Interface Visual)

Gerenciamento visual de **bancos e tabelas**.



O **XAMPP** não é a linguagem, é a **fábrica** que fornece todas as **ferramentas** necessárias para a **aplicação** rodar no seu **computador** (Versão recomendada: PHP 8.2).

⚠ Sintoma (Error)

MySQL fecha inesperadamente ou exibe erro vermelho no painel XAMPP.



🔍 Diagnóstico (Cause)

Conflito de Porta. A porta **3306** já está em uso (ex: Docker, MySQL Workbench, WampServer).



🔧 Solução Técnica (Fix)

1. Acessar MySQL → Config → `my.ini`.
2. Localizar a porta padrão e alterar para **3307**.
3. **Atenção:** Alterar obrigatoriamente em dois blocos: `[client]` e `[mysqld]`.
4. Salvar e reiniciar os serviços.

Impacto no Código: A partir de agora, toda conexão PHP deverá especificar explicitamente a porta **3307**. O phpMyAdmin continua acessível via localhost.

htdocs/sistema_escolar/

| /config/

| /actions/

| /pages/

| /includes/

| /assets/

| | /css/

| | /js/

| | /img/

| index.php



Configurações do sistema (ex: database.php).



Processamento e operações invisíveis (ex: salvar.php).



Páginas visuais da aplicação (ex: cadastro.php).



Componentes UI reutilizáveis para evitar duplicação (Header, Footer, Menu).



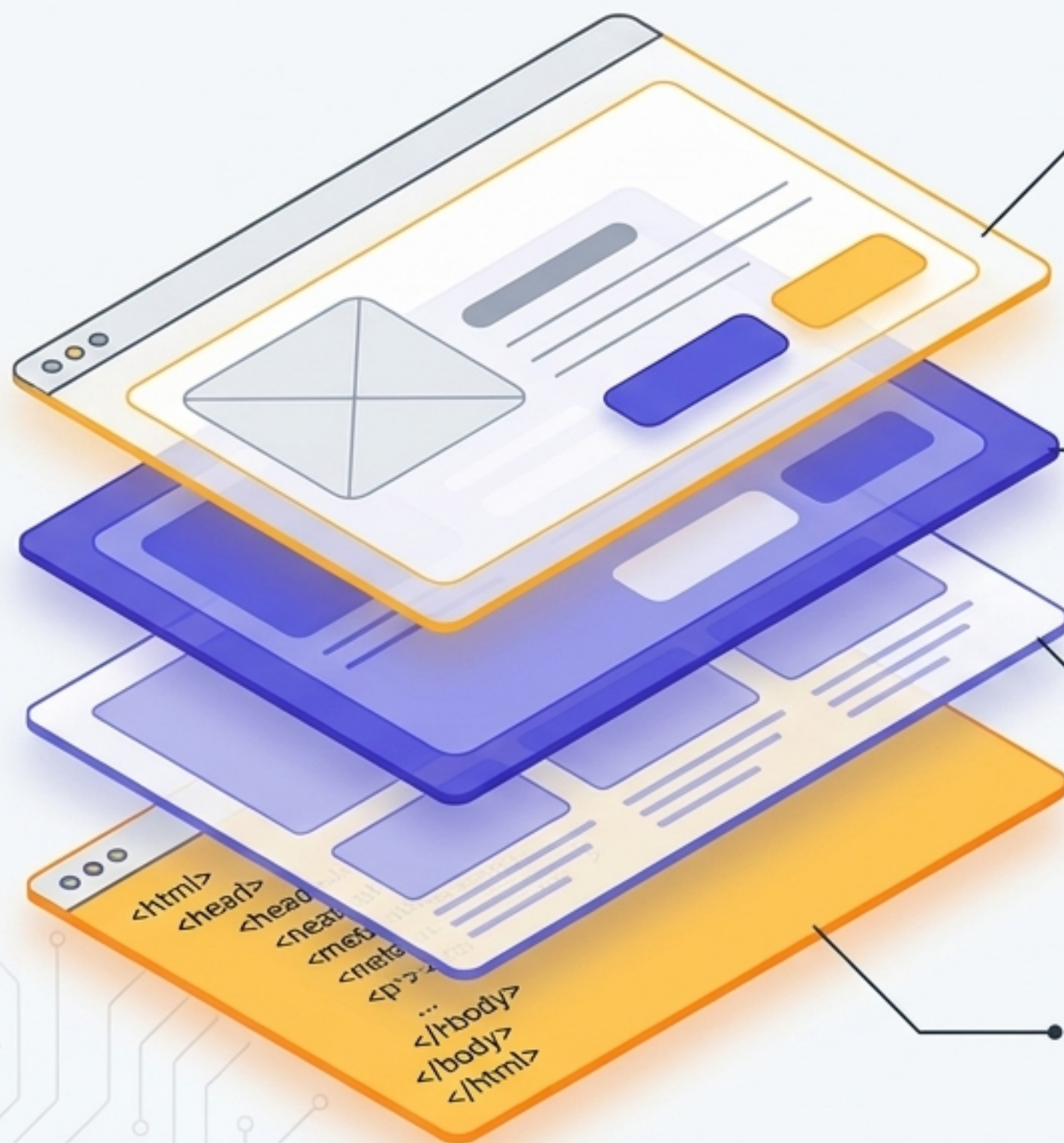
Arquivos estáticos divididos em /css/, /js/ e /img/.



O ponto de entrada principal ou roteador.

Engenharia de Estrutura de Pastas

Separação de Responsabilidades. O erro mais comum de iniciantes é misturar HTML, SQL e PHP em um único arquivo. Modularizar garante manutenção, escalabilidade e facilita o trabalho em equipe.

**JS****A Interatividade (JavaScript):**

Arquivo local (assets/js/script.js) acionado por eventos HTML (onclick).

CSS**O Estilo Customizado (CSS):**

Arquivo local (assets/css/style.css) para regras específicas (ex: bordas arredondadas, cor de fundo).

B**O Framework (Bootstrap 5):**

Importado via CDN. Fornece classes utilitárias (container, card shadow, btn-primary) para layout responsivo imediato.

5**A Base (HTML5):**

Tags de estrutura (<head>, <body>) e Meta Viewport para dispositivos móveis.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Meu Projeto</title>
  <!-- Bootstrap 5 -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.7/dist/css/bootstrap.min.css"
        rel="stylesheet">
  <!-- Fim do projeto -->
  <link href="assets/css/style.css" rel="stylesheet">
</head>

<body>
  <div class="container">
    <div class="card">
      <div class="card-body">
        <div class="text-center">
          <p>Projeto Integrado entre frontend, backend e banco de dados utilizando PHP.</p>
          <button class="btn btn-primary" onclick="alert('Testar JavaScript!')">Testar JavaScript</button>
        </div>
      </div>
    </div>
  </div>

  <!-- Bootstrap 5 -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.7/dist/js/bootstrap.bundle.min.js">
  </script>
  <!-- Fim do projeto -->
  <script src="assets/js/script.js">
  </script>
</body>
</html>
```

Esta organização isola o comportamento da apresentação, preparando um terreno limpo para a injeção futura de dados via PHP.

Conectando ao MySQL: Matriz de Decisão

	mysqli 🏆	PDO 🏆
Compatibilidade	MySQL apenas	Múltiplos bancos de dados
Paradigma	Orientado a Objetos e Procedural	Estritamente Orientado a Objetos
Tratamento de Erros	Básico	Orientado a Exceções try/catch
Flexibilidade	Média	Alta
Segurança	Suporta Prepared Statements	Suporta Prepared Statements

Veredito de Engenharia

O PDO (PHP Data Objects) é o padrão ouro para aplicações modernas devido à sua flexibilidade e segurança por exceções. O mysqli permanece excelente para sistemas legados focados apenas em MySQL.

Anatomia de uma Conexão Segura

Variáveis de Ambiente

Isolamento de credenciais:
'\$host', '\$dbname', '\$user'
(root), '\$password' (vazio),
'\$port' (3307).

Regra de Ouro: Nunca deixar credenciais hardcoded no meio do sistema. Centralize no config/database.php.

```
<?php
$host = "localhost";
$dbname = "sistema_escolar";
$user = "root";
$password = "";
$port = 3307;

try {
    $pdo = new PDO(
        "mysql:host=$host;
        port=$port;
        dbname=$dbname",
        $user,
        $password
    );
    $pdo->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
    );
    echo "Conexão PDO realizada com sucesso";
} catch (PDOException $e) {
    die(
        "Erro de conexão: " .
        $e->getMessage()
    );
}
```

O Bloco Try/Catch

- `new PDO(...)`: Instancia a conexão com a string DSN.
- `setAttribute(...)`: Força o PHP a gritar (lançar exceção) se algo der errado, evitando falhas silenciosas.
- `catch (PDOException $e)`: Captura o erro com segurança, impedindo que o script quebre a tela do usuário.

O Esquema de Dados: Tabela “Alunos”

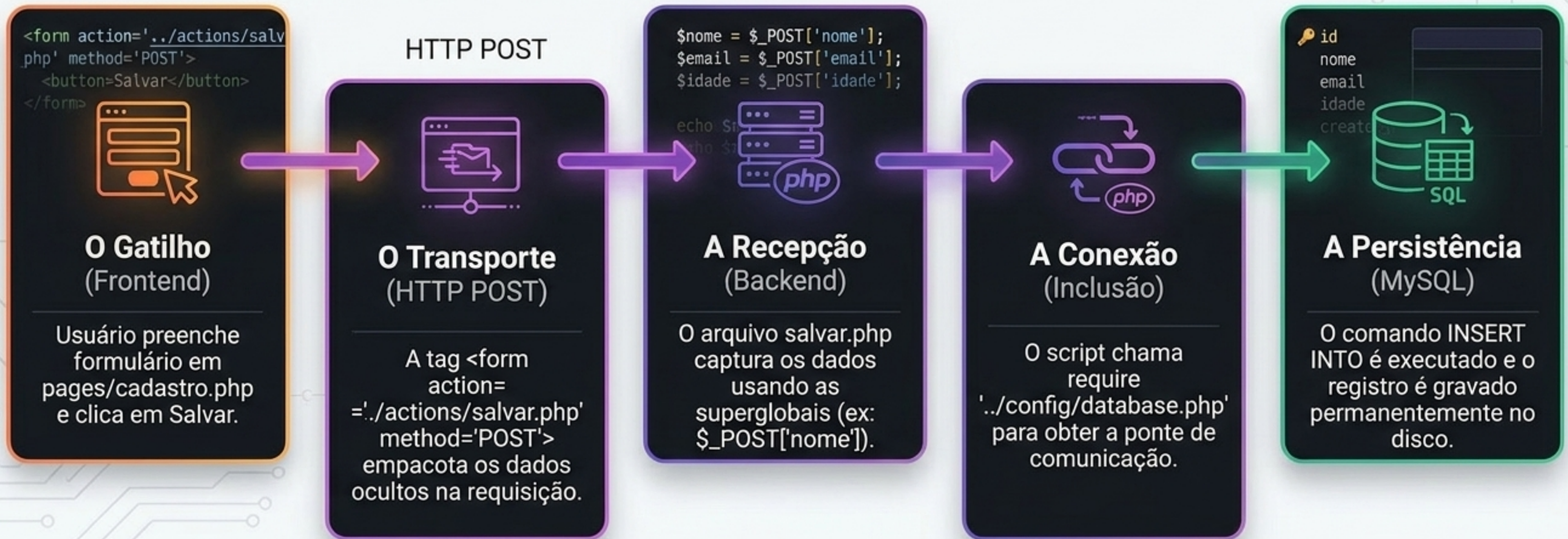
Tabela: **Alunos**

 id	<code>INT AUTO_INCREMENT PRIMARY KEY</code> O identificador único absoluto.
nome	<code>VARCHAR(255) NOT NULL</code> Texto.
email	<code>VARCHAR(255) NOT NULL</code> Texto.
idade	<code>INT NOT NULL</code> Número Inteiro.
created_at	<code>TIMESTAMP</code> Auditoria temporal automática.

```
CREATE TABLE alunos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  idade INT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Contexto: Esta tabela viverá no banco sistema_escolar criado via phpMyAdmin. Ela será o destino final de todas as operações CRUD (Create).

O Fluxo de Integração (CRUD) de Ponta a Ponta



Código Vulnerável

```
<?php
require '../config/database.php';

$nome = $_POST['nome'];
$email = $_POST['email'];
$idade = $_POST['idade'];

$sql = "INSERT INTO alunos (nome, email, idade) VALUES ('$nome', '$email'

if ($conn->query($sql)) {
    echo "Aluno cadastrado com sucesso";
} else {
    echo "Erro ao cadastrar";
}
$conn->close();
```

⚠ O Risco (SQL Injection)

Problema: Concatenar strings diretamente permite que usuários mal-intencionados injetem comandos SQL destrutivos pelo formulário.

Código Seguro

```
<?php
require '../config/database.php';

$nome = $_POST['nome'];
$email = $_POST['email'];
$idade = $_POST['idade'];

$stmt = $conn->prepare(
    "INSERT INTO alunos(nome, email, idade) VALUES (?, ?, ?)"
);

$stmt->bind_param(
    "ssi",
    $nome,
    $email,
    $idade
);

if ($stmt->execute()) {
    echo "Aluno cadastrado com sucesso";
} else {
    echo "Erro ao cadastrar";
}
$stmt->close();
$conn->close();
```

✅ O Padrão Ouro (Prepared Statements)

1. **Prepare:** `$conn->prepare(...)` Avisa o banco sobre o comando, usando placeholders (?).
2. **Bind:** `$stmt->bind_param('ssi', ...)` Vincula as variáveis aos tipos estritos: s=string, i=integer.
3. **Execute:** `$stmt->execute()` Executa com segurança isolada.

Atividade Prática: O Desafio de Integração

Objetivo Core: Desenvolver o fluxo completo ponta-a-ponta (Frontend → Backend → Banco de Dados) aplicando os conceitos de Engenharia de Software.

Os Temas (Escolha um):



1. Cadastro de Alunos



2. Cadastro de Produtos



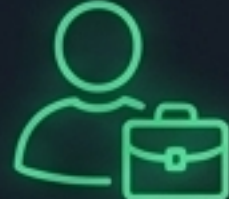
3. Cadastro de Funcionários



4. Cadastro de Cursos



5. Cadastro de Livros



6. Cadastro de Clientes

Requisitos Técnicos do Sistema

Frontend

- ✓ Uso obrigatório de HTML5, CSS e Bootstrap.
- ✓ Organização visual profissional.
- ✓ Formulário funcional com no mínimo 3-4 campos (ex: nome, preço, categoria).

Backend & Arquitetura

- ✓ Estrutura de pastas modular (pages/, config/, actions/, etc).
- ✓ PHP processando dados via \$_POST.
- ✓ Código rigorosamente documentado com comentários explicando a lógica.

Banco de Dados

- ✓ Conexão segura (PDO ou MySQLi).
- ✓ Tabela mapeada com a regra de negócio escolhida.
- ✓ Persistência de dados (INSERT) e listagem dos registros.

Fluxo de Entrega Profissional (Git e GitHub)



Aviso Strict

Projetos sem PR, sem comentários, ou em repositórios públicos não serão aceitos.



Desenvolva. Conecte. Evolua.

O desenvolvimento de software bem-sucedido vai muito além de fazer o código funcionar na sua máquina. Trata-se de construir sistemas modulares, escaláveis e seguros desde o primeiro diretório criado.

Organize sua arquitetura. **Valide** suas entradas. **Proteja** suas *queries*. A engenharia começa agora.